



AgroSense_Submersible Water Level Sensor LoRaWAN® Manual V1.0

Author: Yuki

Time: 2026.06.15

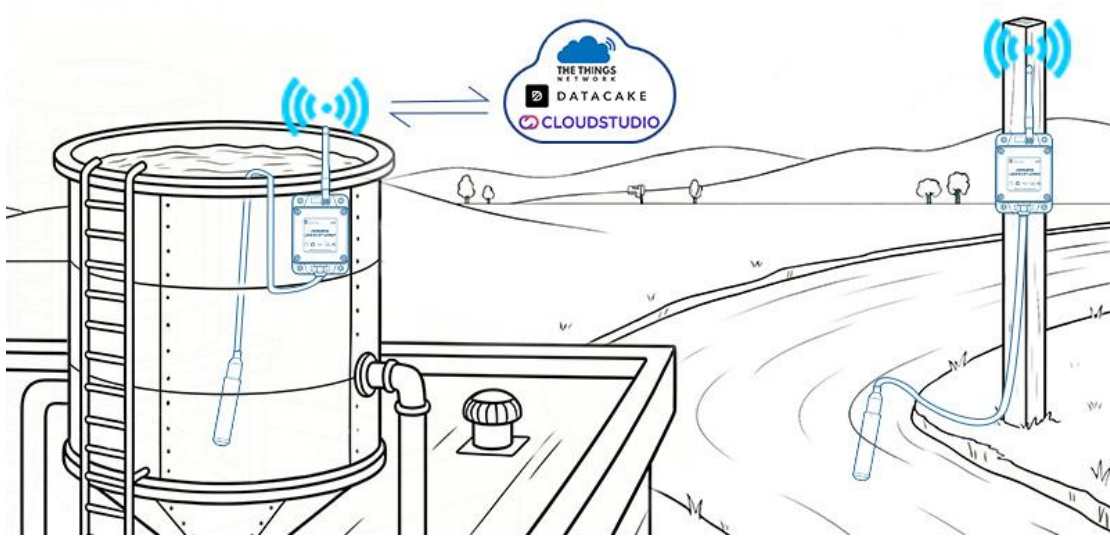
Contents

1 Product Description	1
1.1 Introduction	1
1.2 Feature	2
1.3 Parameter	3
2 Technical route	4
2.1 System Framework	4
2.2 Regional frequency band	5
3 Usage	6
3.1 TTN and ThingSpeak	6
3.2 Datacake	15

1 Product Description

1.1 Introduction

The AgroSense Submersible Water Level sensor is used for tank level or open water applications measuring while fully submerged in liquid over the LoRaWAN® network. It detects the liquid level, with cloud service such as DataCake/Thingspeak, it get easy for remote monitoring/alarming water level stats.



This product is based on LoRaWAN® V1.0.3 OTAA Class A, It uplink the water level (0~5m) every 1 hours, Users can send downlink commands from cloud server, to change the update interval to data uplink, from 5 minutes to 24 hours.

This Module supports global frequency bands (EU868/US915) and works seamlessly with platforms such as TTN, Datacake, and CloudStudio to enable data upload and remote control.

Besides, it has Local storage for max 3,300+ records, ensures critical data is retained during connectivity gaps.



1.2 Feature

- Includes a **high precision** sensor.
- Compatible with Worldwide **LoRaWAN® Networks**: Support the universal frequency bands EU868/ US915.
- LoRaWAN version: LoRaWAN Specification 1.0.3.
- **Long Range**: Up to 2 kilometers in the city, up to 10 kilometers in the wilderness, receive sensitivity -137dBm , transmit power up to 21dBm.
- **Ultra-low power** consumption design, traditional AAA alkaline dry battery can be used for one year.
- **Data encryption**: Provide end-to-end secure communication, including device authentication and network data encryption, to ensure the security of data transmission and prevent data theft and malicious attacks.
- **High stability and reliability**: good stability in noisy environments, able to penetrate buildings and obstacles, so it can maintain good communication quality in urban and suburban environments.
- Suitable for **Harsh Environments**: Can work normally under the temperature of $-40^{\circ}\text{C} \sim 85^{\circ}\text{C}$, IP68 waterproof, suitable for outdoor use in harsh conditions, high UV, dusty, heavy rain and other bad weather.
- Monitor data and upload **real-time** data regularly.
- Modify the product parameters through **AT commands**.

1.3 Parameter

1. General Parameters

Product Model	AGWLS010
Range	0 ~ 5m (Customizable up to 500 m)
Accuracy	± 0.5% FS
Resolution	0.01 m
Long-term Stability	≤ ± 0.3% FS/year
Cable Length	5 m

2. Wireless Parameters

Communication Protocol	Standard LoRaWAN® protocol V1.0.3
Network Access/Operating Mode	OTAA Class A
MAX Transmit Power	22dBm
Receiver Sensitivity	-137dBm/125kHz SF=12
Frequency Band	EU868/US915

3. Physical Parameters

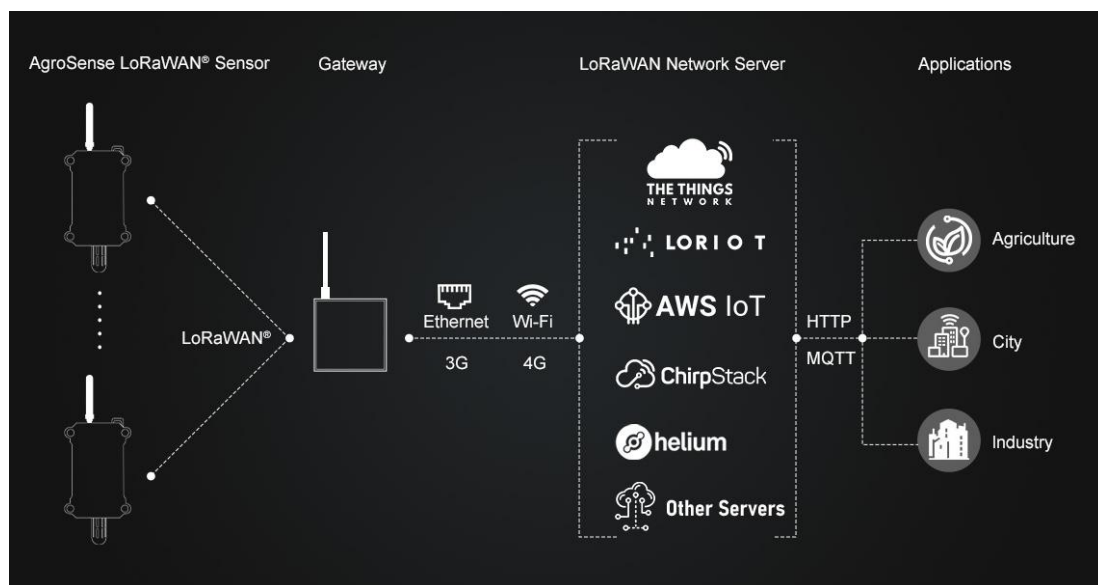
Power Supply	1 x ER34615 3.6V Lion batteries 19000 maH
Operating Temperature	-20°C ~55°C
Level Sensor	IP68
Protection Class	IP67
Main Unit	86*86*57mm
Level Sensor	145 × φ25 mm
Mounting	Pole, Wall

2 Technical route

2.1 System Framework

AgroSense_Submersible Water Level Sensor uses LoRAWAN technology, and its network architecture includes four parts: End Nodes, Concentrator/Gateway, Network Server and Application Server.

End Nodes	It is responsible for collecting sensing data and then transmitting it to Gateway via the LoRaMAC protocol.
Concentrator/Gateway	It is mainly responsible for transmitting node data to the server.
Network Server	Organize the data into JSON packets and decode them.
Application Server	Display the data.



The steps to achieve the detection of leak is:

1. Collect ADC voltage from sensors, and send the data from End Node to Gateway.
2. The Gateway packages node data and transmits it to the Network Server.
3. The Network Server decodes the data and sends it to the Applications.
4. Finally, user can monitor the data in the APP.

2.2 Regional frequency band

At the present moment, our product solely accommodates compatibility with the US915 and EU868.

area	frequency band	center frequency
China	470-510MHz	CN486MHz
America	902-928MHz	US915MHz
Europe	863-870MHz	EU868MHz
Korea	920-923MHz	KR922MHz
Australia	915-928MHz	AU923MHz
New Zealand	921-928MHz	NZ922MHz
Asia	920-923MHz	AS923MHz

3 Usage

We use The Things Network or Datacake as our Network Server, we need to configuration the country/ area frequency, inputting DEV EUI/ APP EUI/ APP Key, decodes, and connect to ThingSpeak or Datacake.

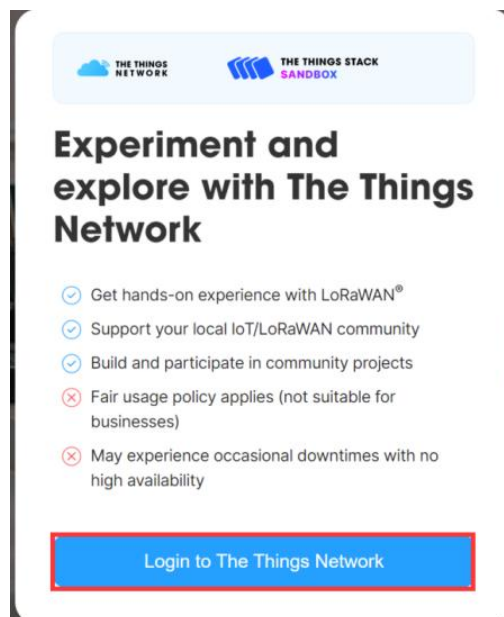
DEV EUI	Unique identification of device, authorized by IEEE
APP EUI	Unique identification of application
APP Key	One of the join network parameters on OTAA mode, calculated by DE EUI

- End Nodes and Gateway: AgroSense_Submersible Water Level Sensor.(The AgroSense series is applicable)
- Network Server: The Things Network. (Loriot, AWS IoT, ChirpStack, ect)
- Application Server: ThingSpeak.(Datacake, Blockbax, akenza, ect)

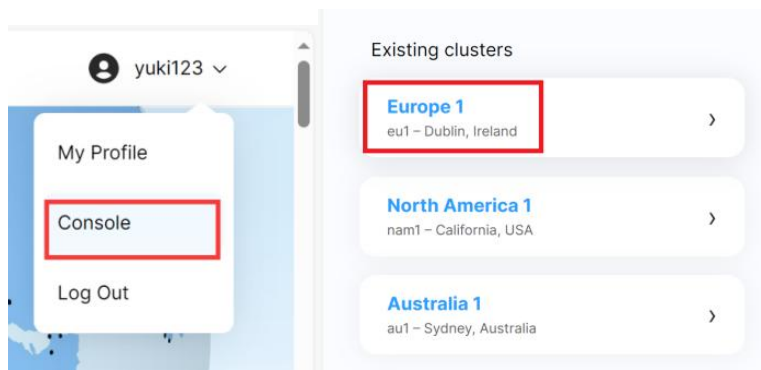
3.1 TTN and ThingSpeak

3.1.1 Network Server configuration

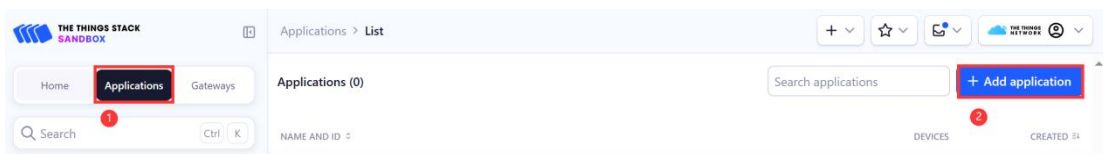
- Open The Things Network in your browser and login it. (Or register an account)



- Click “Console” and select clusters. (we take the European region for example.)



- Click “Go to applications” --> “+ Create application”.



- Write the Application ID and click “Create application”.

Application ID *

agrosense-sensor

Application name

My new application

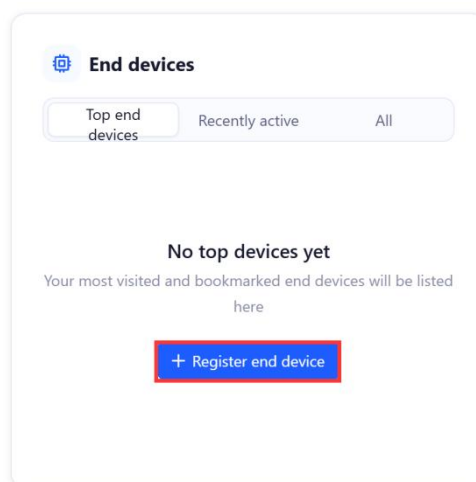
Description

Description for my new application

Optional application description; can also be used to save notes about the :

Create application

- Click “+ Register and device”.



- Following the steps, and input the DEV EUI/ APP EUI/ APP Key (notice: JoinEUI=APP EUI) and subsequently click on "Register end device" to complete the registration process.

End device type

Input method ⓘ

☐ Select the end device in the LoRaWAN Device Repository

☒ Enter end device specifics manually **1**

Frequency plan ⓘ *

Europe 863-870 MHz (SF9 for RX2 - recommended) | v

LoRaWAN version ⓘ *

LoRaWAN Specification 1.0.3 | v

Regional Parameters version ⓘ *

RP001 Regional Parameters 1.0.3 revision A | v **2**

[Show advanced activation, LoRaWAN class and cluster settings](#)

Provisioning information

JoinEUI ⓘ *

48 FF 00 00 00 00 01 65 **3** Confirm **4**

Continue, please enter the JoinEUI **4** and device so we can determine onboarding options

Provisioning information

JoinEUI ⓘ *

48 FF 00 00 00 00 01 65 Reset

This end device can be registered on the network

DevEUI ⓘ *

48 E6 63 FF FE 30 01 65 Generate 0/50 used

AppKey ⓘ *

4A 35 62 6B 95 AB 5B 4D 3F 3B DE 12 71 B1 6F 2A Generate

End device ID ⓘ *

eui-48e663ffe300165

This value is automatically prefilled using the DevEUI

After registration

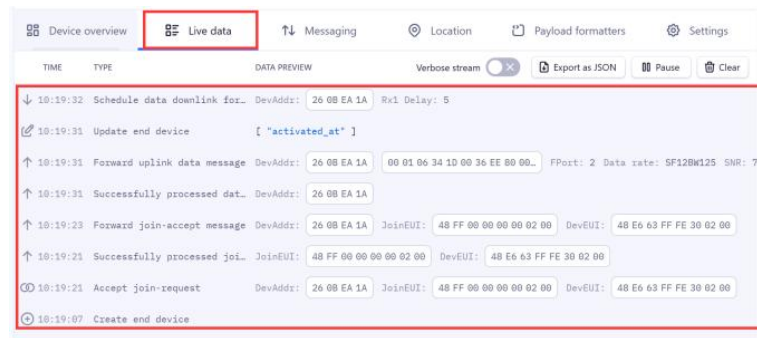
☒ View registered end device

☐ Register another end device of this type

Register end device

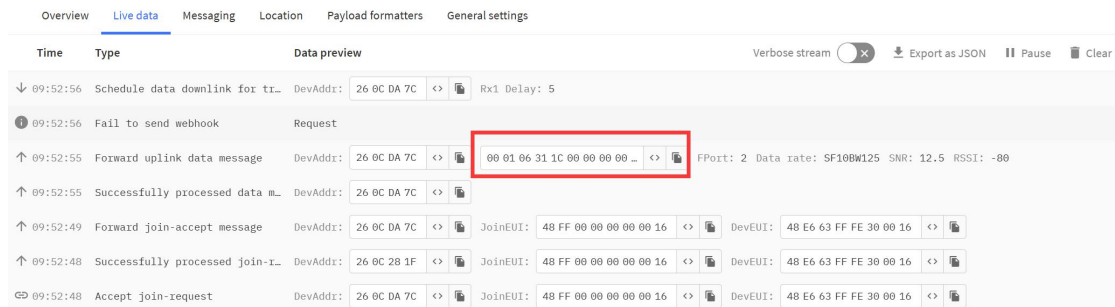


- Plug the battery and press RES button, you can see the device is connected successfully in the TTN.



3.1.2 Decoder

- Now, we need to decode the data.



Data length	Data description	Value range	Explanation
byte 0	Data packet sequence number high 8 bits	0-0xFFFF	Counting starts from 0 and increments, resetting back to 0 after reaching 65535
byte 1	Data packet sequence number low 8 bits		
byte 2	Battery voltage		The value is obtained by amplifying the data by 10 times, and the actual value needs to be divided by 10 to convert to the actual battery voltage. The purpose of multiplying by 10 is to retain one decimal place of the voltage value. For example, if the value is 0x21 = 33, then the battery voltage is 3.3V.
byte 3	Water_bat bits 8 to 15		ADC voltage of the 4–20mA water level sensor after the 150Ω resistor conversion. Unit: mV.
byte 4	Water_bat bits 0 to 7		
byte 5	Data transmission interval bits 24 to 31		Uplink data transmission interval. The unit is seconds. For example, if the value is 0x0000012C (300), then the transmission interval is 300 seconds (5 minutes).
byte 6	Data transmission interval bits 16 to 23		
byte 7	Data transmission interval bits 8 to 15		

byte 8	Data transmission interval bits 0 to 7		
byte 9	Timestamp bits 24 to 31		These 4 bytes form a 32-bit Unix timestamp. It represents the number of seconds elapsed since 1970-01-01 00:00:00 (UTC) when this data record was generated. After combining the 4 bytes, the value can be converted into the corresponding date and time.
byte 10	Timestamp bits 16 to 23		
byte 11	Timestamp bits 8 to 15		
byte 12	Timestamp bits 0 to 7		
byte 13	System reserved bits	0	Always 0 (and can be ignored)

Example: 0x00, 0x55, 0x21, 0x02, 0xA8, 0x00, 0x36, 0xEE, 0x80, 0x69, 0xCD, 0xCD, 0xED, 0x00

Data parsing:

Battery voltage is 3.3 V.

Water bat 680 mV.(Converted to a depth of 18.3 cm, For details on this conversion process, please refer to the [data parsing.](#))

Data transmission interval value is 3600s.

Timestamp value is 1775095277 (2026-04-02 02:01:17).

- Know how to decode it after, we need to write it in code. (you can check it out on [Github](#))

```
function decodeUplink(input) {
    var num = input.bytes[0] * 256 + input.bytes[1]
    var bat = input.bytes[2] / 10.0
    var water_bat = input.bytes[3] * 256 + input.bytes[4]    //mv
    var current_mA = water_bat / 150.0;    //res: 150Ω
    var Water_depth = ((current_mA - 4.0) / 16.0) * 5.0;    //4~20mA -> 0~5m

    if (Water_depth < 0) {
        Water_depth = 0;
    }
    if (Water_depth > 5) {
        Water_depth = 5;
    }

    var Water_pressure = 1000 * 9.8 * Water_depth;    // p = ρgh, ρ = 1000kg/m³, g = 9.8m/s²
    Water_depth = Number(Water_depth.toFixed(2));
    Water_pressure = Number(Water_pressure.toFixed(2));
    var interval = (input.bytes[5] * 16777216 + input.bytes[6] * 65536 + input.bytes[7] * 256 + input.bytes[8]) /
1000    //s
    var time = (input.bytes[9]*16777216 + input.bytes[10]*65536 + input.bytes[11]*256 + input.bytes[12])
    var timeStr = null;
    var d = new Date(time * 1000)
    timeStr =
```

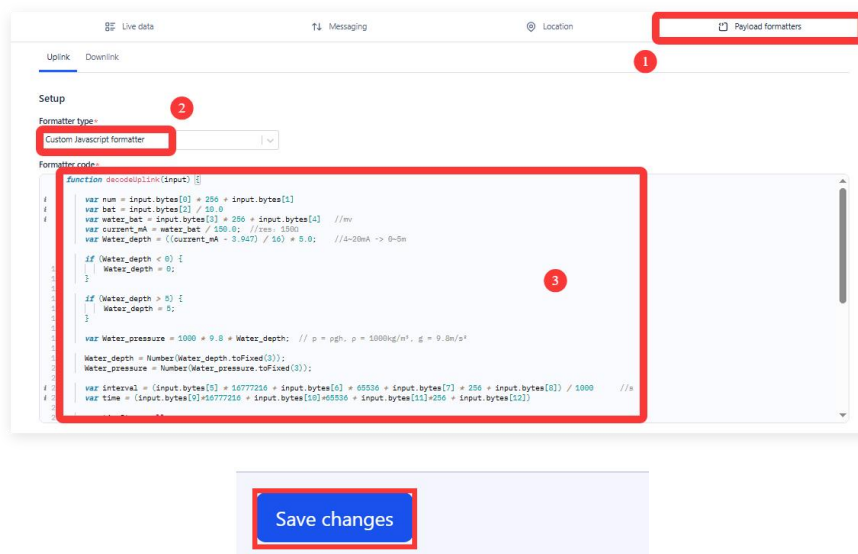
```

d.getUTCFullYear() + "-" +
String(d.getUTCMonth() + 1).padStart(2, "0") + "-" +
String(d.getUTCDate()).padStart(2, "0") + " " +
String(d.getUTCHours()).padStart(2, "0") + ":" +
String(d.getUTCMinutes()).padStart(2, "0") + ":" +
String(d.getUTCSeconds()).padStart(2, "0");

return {
  data: {
    field0: num,
    field1: bat,
    field2: Water_depth, //unit: m
    field3: Water_pressure, //unit: Pa
    field4: interval,
    device_time: timeStr,
    device_time_unix: time
  },
};
}

```

- Select “Payload formatters” and follow the steps.



3.1.3 Application Server configuration

In the Application Server configuration, we need to create ThingSpeak channel and get Channel ID and API Key, this is the key to our connection to TTN.

- Login to the ThingSpeak. (Or register an account)



MathWorks®

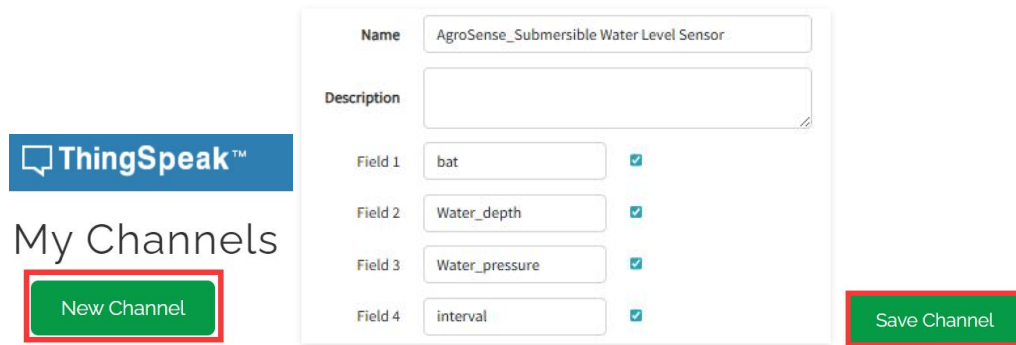
Email

No account? [Create one!](#)

By signing in, you agree to our [privacy policy](#).

Next

- Click “New Channel”, fill in the Channel name and field names and click “Save Channel”.



ThingSpeak™

My Channels

New Channel

Name: AgroSense_Submersible Water Level Sensor

Description:

Field 1: bat ☒

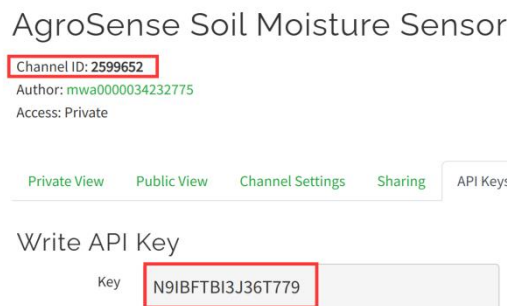
Field 2: Water_depth ☒

Field 3: Water_pressure ☒

Field 4: interval ☒

Save Channel

- After successful creation, copy the Channel ID and API Key.



AgroSense Soil Moisture Sensor

Channel ID: 2599652

Author: mwa0000034232775

Access: Private

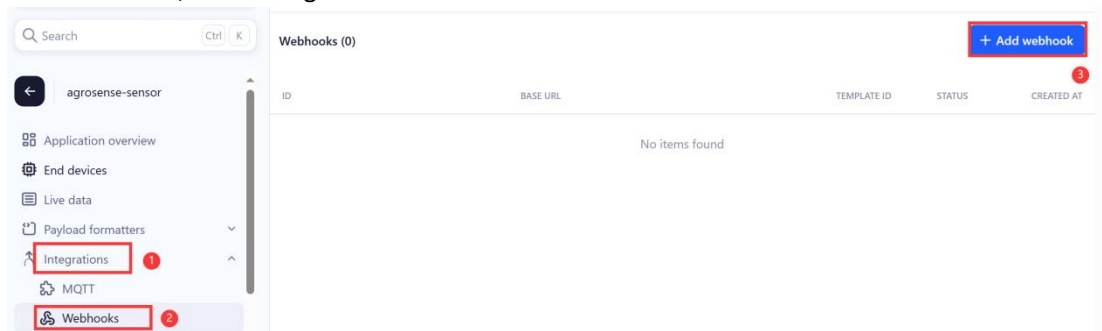
Private View Public View Channel Settings Sharing API Keys

Write API Key

Key: N9IBFTBI3J36T779

3.1.4 Connect the Network Server and Application Server

- In the TTN, click “integrations” --> “Webhooks” --> “+ Add webhook”.



Search Ctrl K

agrosense-sensor

Application overview

End devices

Live data

Payload formatters

Integrations ¹

MQTT

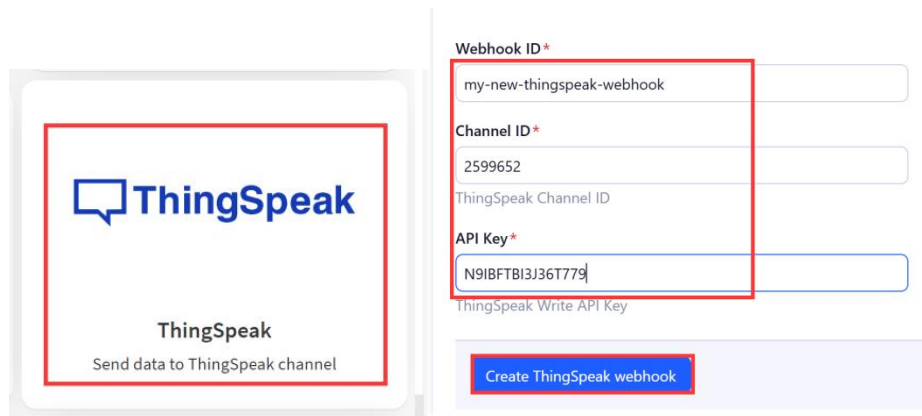
Webhooks ²

Webhooks (0)

+ Add webhook ³

ID	BASE URL	TEMPLATE ID	STATUS	CREATED AT
No items found				

- Select “ThingSpeak”, Fill in the Webhook ID and paste the Channel ID and API Key, click “Create ThingSpeak Webhook”.



Webhook ID *

my-new-thingspeak-webhook

Channel ID *

2599652

ThingSpeak Channel ID

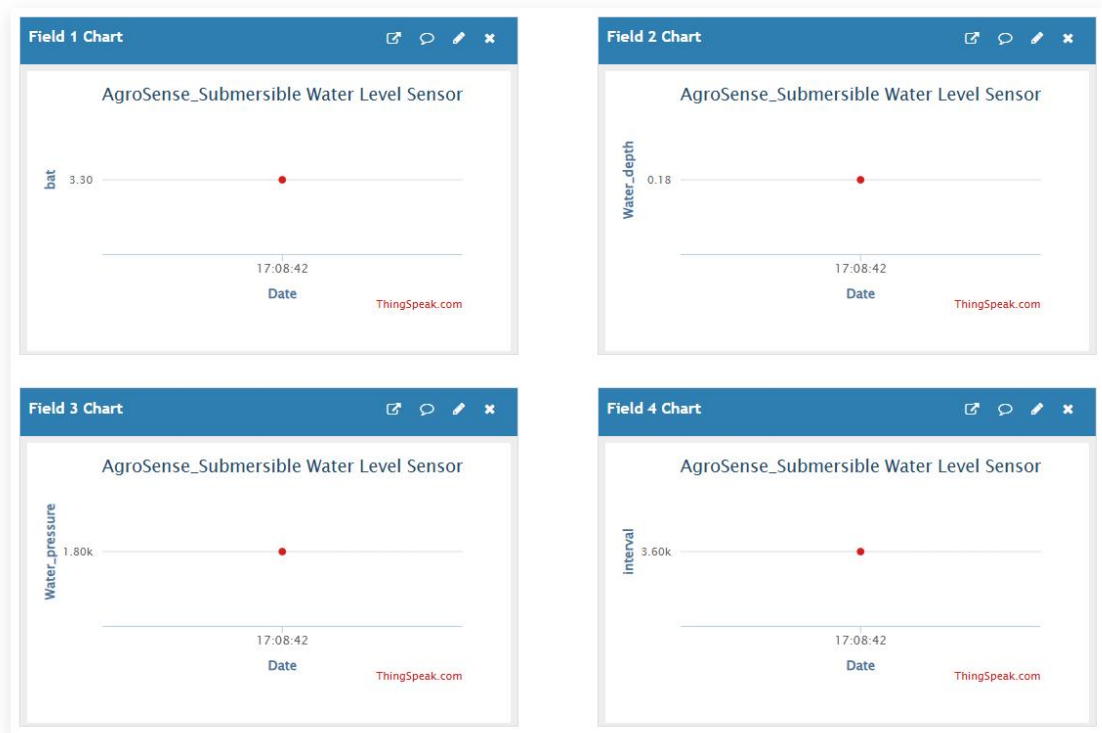
API Key *

N9IBFTB13J36T779

ThingSpeak Write API Key

Create ThingSpeak webhook

- Press RST button, wait about a minute, you will successfully see the data in ThingSpeak.



3.1.5 Downlink

- Modification time interval (Fport1)

Modify the time interval for uploading data, the default is one hour.

1、 If you need to change time Interval (Default 12 hours), you can click “Payload formatters-->Downlink” and follow the steps.

Formatter code you can find in [Github](#).

Device overview Live data Messaging Location Payload formatters

Uplink: Downlink 2

Setup

Formatter type* Custom Javascript formatter 3

Formatter code*

```

1 // Encoder function to be used in the TTN console for downlink payload
2 function Encode(input) {
3   var minutes = input.minutes;
4   // Converting minutes to seconds
5   var seconds = minutes * 60;
6   // If the number of seconds is less than 300 seconds, set it to 300 seconds
7   if (seconds < 300) {
8     seconds = 300;
9   }
10  var payload = [
11    (seconds >> 24) & 0xFF,
12    (seconds >> 16) & 0xFF,
13    (seconds >> 8) & 0xFF,
14    seconds & 0xFF
15  ];
16  return payload;
17 }
18
19
20
21

```

4

2、Click “Save changes”.

Save changes

3、Click “Messaging-->Schedule downlink”.

Note: you must use this format:

```

{
  "minutes": 60
}

```

Device overview Live data Messaging

Schedule downlink Simulate uplink

Schedule downlink

Insert Mode

☒ Replace downlink queue

☐ Push to downlink queue (append)

FPort*

1

Payload type

☐ Bytes

☒ JSON

Payload

```

{
  "minutes": 5
}

```

The decoded payload of the downlink message

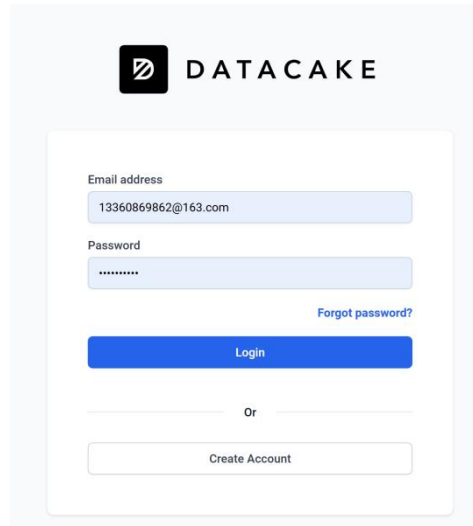
☐ Confirmed downlink

Schedule downlink

4、The modified interval will be updated after the next data upload.

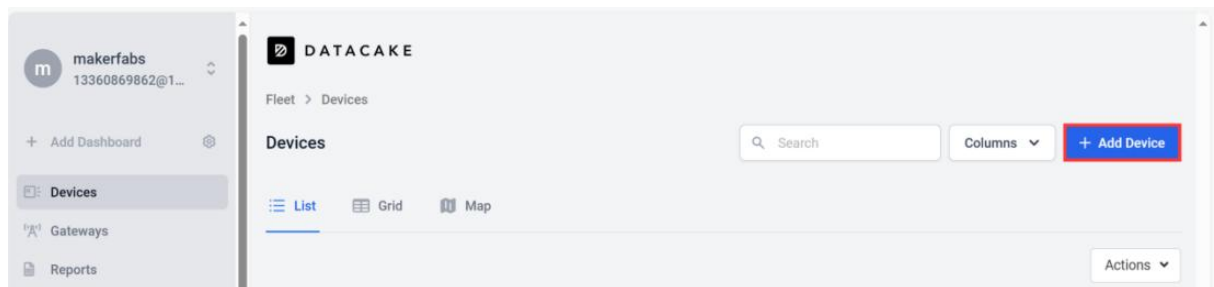
3.2 Datacake

1、Login datacake or Create Account

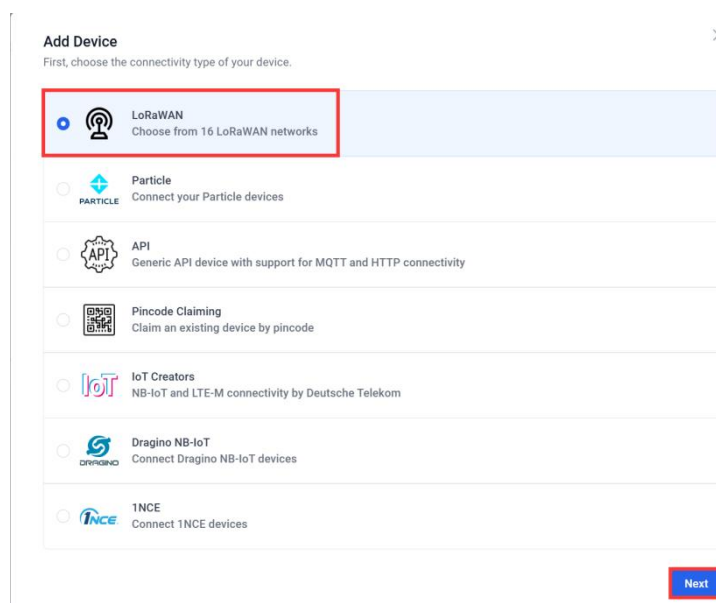


The image shows the Datacake login and registration interface. At the top is the Datacake logo. Below it is a form with two input fields: 'Email address' containing '13360869862@163.com' and 'Password' with masked characters. A 'Forgot password?' link is next to the password field. Below the fields is a blue 'Login' button. Underneath is an 'Or' separator, followed by a 'Create Account' button.

2、Click “Add Device”

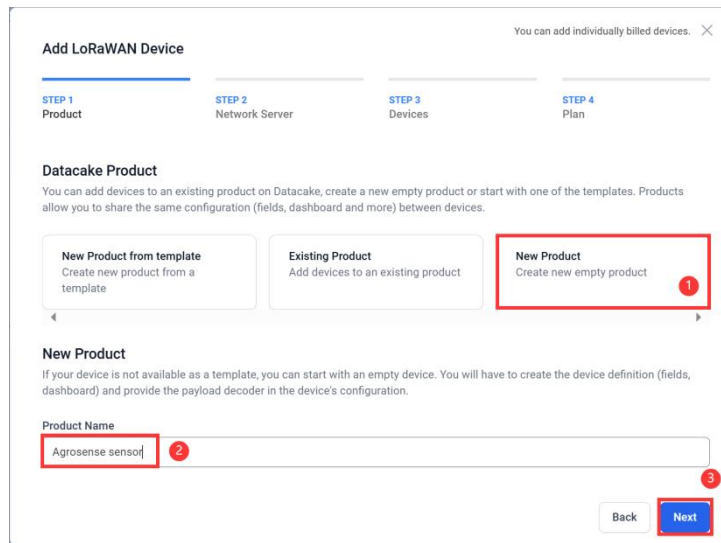


3、Select LoRaWAN and click “Next”

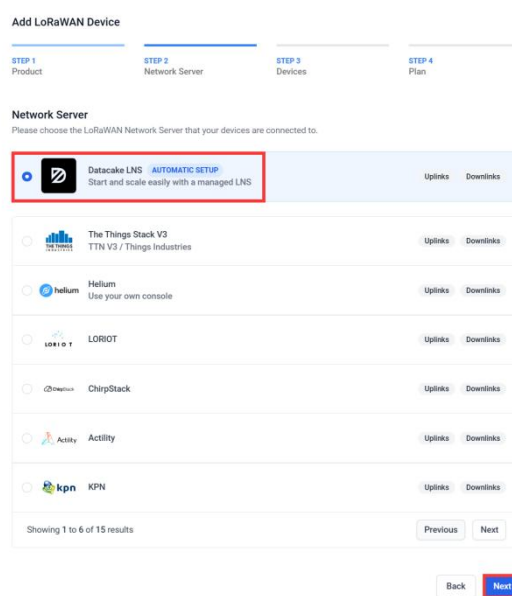


The image shows the 'Add Device' dialog box. It has a title 'Add Device' and a subtitle 'First, choose the connectivity type of your device.' Below this is a list of connectivity options. The first option, 'LoRaWAN', is selected and highlighted with a red box. It includes a radio button, an antenna icon, and the text 'Choose from 16 LoRaWAN networks'. Other options include 'Particle', 'API', 'Pincode Claiming', 'IoT Creators', 'Dragino NB-IoT', and '1NCE'. A red-bordered 'Next' button is at the bottom right.

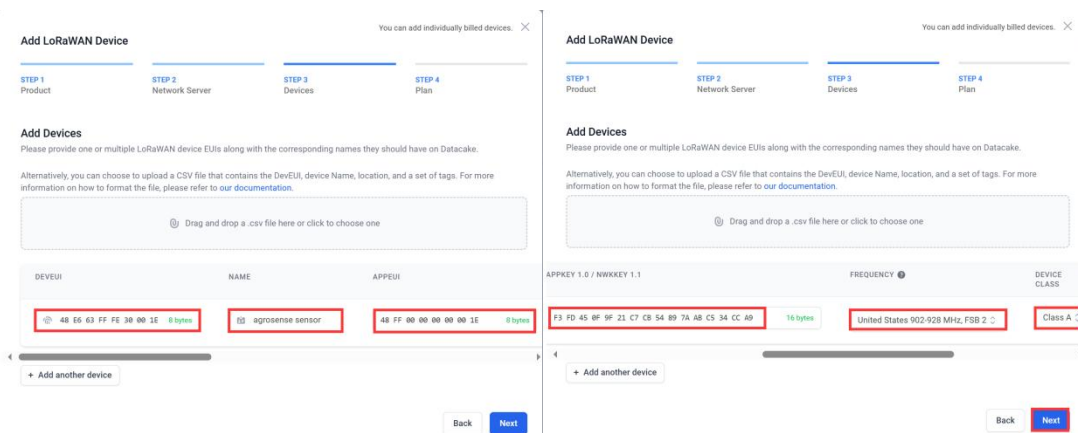
4、Select a Product based on your needs, take "Create new empty product" as an example.



5、Select "Datacake LNS"



6、Enter DEVEUI、APPEUI、APPKEY、FREQUENCY (take 915 for example) and DEVICE CLASS.



7、Choose the type according to your needs, and click “Add 1 device”.

8、Click to go to the device you just added.

DEVICE	PRIMARY	SECONDARY	DEVICE SIGNAL	DEVICE BATTERY
AgroSense_Air Temperature and Humidity Sensor	40.2	25	-48	2.5
AgroSense-carbon dioxide (CO2) sensor	0	N/A	-86	3.3
AgroSense sensor	N/A	N/A	N/A	N/A

9、Click “Configuration”, enter Decoder and click “Save”.(You can check it out on [Guihub](#))

```

function decode(payload, port) {
    var input = {
        bytes: payload
    };
    // var hex = input.bytes[0] * 256 + input.bytes[1];
    // var hex = input.bytes[2] * 256 + input.bytes[3];
    // var water_level_flag = input.bytes[4];
    // var water_level_val = input.bytes[5];
    // var water_level_time = input.bytes[6] * 1000000 + input.bytes[7] * 10000 + input.bytes[8] * 1000 + input.bytes[9] * 100 + input.bytes[10];

    var decoded = {
        hex: hex,
        water_level_flag: water_level_flag, // Normal is 0, Inverse is 1.
        water_level_val: water_level_val,
        water_level_time: water_level_time
    };

    // Test for LoRa properties in normalizedPayload
    try {
        console.log('normalizedPayload', normalizedPayload); // log to check normalizedPayload structure
    } catch (error) {
        console.log('Error occurred while decoding LoRa properties: ' + error);
    }

    return {
        payload: "hex", value: decoded.hex,
        flag: "water_level_flag", value: decoded.water_level_flag,
        val: "water_level_val", value: decoded.water_level_val,
        time: "water_level_time", value: decoded.water_level_time
    };
}
    
```

```

function Decoder(payload, port) {

    var input = {

        bytes: payload

    };

    var num = input.bytes[0] * 256 + input.bytes[1]

    var bat = input.bytes[2] / 10.0

    var water_bat = input.bytes[3] * 256 + input.bytes[4]    //mv

    var current_mA = water_bat / 150.0;    //res: 150 Ω

    var Water_depth = ((current_mA - 4.0) / 16.0) * 5.0;    //4~20mA -> 0~5m

    if (Water_depth < 0) {

        Water_depth = 0;

    }

    if (Water_depth > 5) {

        Water_depth = 5;

    }

    var Water_pressure = 1000 * 9.8 * Water_depth;    // p = ρ gh, ρ = 1000kg/m³, g = 9.8m/s²

    var interval = (input.bytes[5] * 16777216 + input.bytes[6] * 65536 + input.bytes[7] * 256 + input.bytes[8]) /
1000    //s

    var time = (input.bytes[9]*16777216 + input.bytes[10]*65536 + input.bytes[11]*256 + input.bytes[12])

    var decoded =

    {

        NUM:num,

        BAT:bat,

        WATER_DEPTH:Water_depth,

        WATER_PRESSURE:Water_pressure,

        INTERVAL:interval,

        time: time

    };

    // Test for LoRa properties in normalizedPayload

    try {

```

```

    if (normalizedPayload.gateways && normalizedPayload.gateways.length > 0) {

        decoded.LORA_RSSI = normalizedPayload.gateways[0].rssi || 0;

        decoded.LORA_SNR = normalizedPayload.gateways[0].snr || 0;

    } else {

        decoded.LORA_RSSI = 0;

        decoded.LORA_SNR = 0;

    }

    decoded.LORA_DATARATE = normalizedPayload.spreading_factor

        || normalizedPayload.data_rate

        || (normalizedPayload.networks && normalizedPayload.networks.lora &&
normalizedPayload.networks.lora.dr)

        || "unknown";

    } catch (error) {

        console.log('LoRa property parsing error:', error);

        decoded.LORA_RSSI = 0;

        decoded.LORA_SNR = 0;

        decoded.LORA_DATARATE = "unknown";

    }

    return [

        { field: "BAT", value: decoded.BAT, timestamp: time },

        { field: "WATER_DEPTH", value: decoded.WATER_DEPTH, timestamp: time },

        { field: "WATER_PRESSURE", value: decoded.WATER_PRESSURE, timestamp: time },

        { field: "INTERVAL", value: decoded.INTERVAL, timestamp: time },

        { field: "LORA_RSSI", value: decoded.LORA_RSSI, timestamp: time },

        { field: "LORA_SNR", value: decoded.LORA_SNR, timestamp: time },

        { field: "LORA_DATARATE", value: decoded.LORA_DATARATE, timestamp: time }

    ];

}

```

10、Follow the steps to add a field.

Fields

Fields describe the data the device will store.

+ Add Field

Add Field

the schema of the data the device stores.

Type

Numeric

Name

Bat

Identifier

BAT

The field identifier is a unique string that can consist of uppercase letters, numbers and underscores. Once a field has been created, the identifier can not be changed.

Unit

Optional

Role

None

Assign roles to highlight key measurement fields across the platform; each role is limited to one field per product.

Semantic

None

Assign semantic fields to standardize data across devices.

Formula

Formulas can be used to perform calculations on values from other fields. Fields that have a formula can not be written to from a decoder or via the API.

Optional

Use Formula

Cancel

Add Field

Add Field

the schema of the data the device stores.

Type

Numeric

Name

Water Depth

Identifier

WATER_DEPTH

The field identifier is a unique string that can consist of uppercase letters, numbers and underscores. Once a field has been created, the identifier can not be changed.

Unit

Optional

Role

None

Assign roles to highlight key measurement fields across the platform; each role is limited to one field per product.

Semantic

None

Assign semantic fields to standardize data across devices.

Formula

Formulas can be used to perform calculations on values from other fields. Fields that have a formula can not be written to from a decoder or via the API.

Optional

Use Formula

Cancel

Add Field

Add Field

the schema of the data the device stores.

Type

Numeric

Name

Water Pressure

Identifier

WATER_PRESSURE

The field identifier is a unique string that can consist of uppercase letters, numbers and underscores. Once a field has been created, the identifier can not be changed.

Unit

Optional

Role

None

Assign roles to highlight key measurement fields across the platform; each role is limited to one field per product.

Semantic

None

Assign semantic fields to standardize data across devices.

Formula

Formulas can be used to perform calculations on values from other fields. Fields that have a formula can not be written to from a decoder or via the API.

Optional

Use Formula

Cancel

Add Field

11、 Press RST button, wait until the sensor connects to the gateway successfully, you will see the data the sensor is currently reading.

Live data

NAME	IDENTIFIER	TYPE	ROLE / SEMANTIC	CURRENT VALUE	LAST UPDATE
Bat	BAT	Numeric		0	3 minutes ago
Water Depth	WATER_DEPTH	Numeric		0	2 minutes ago
Water Pressure	WATER_PRESSURE	Numeric		0	15 seconds ago

12、 To get a better look at the data, we can add widget.

Click “Dashboard-->switch-->+ Add Widget”.

DATA CAKE

Fleet > agrosense sensor

agrosense sensor

Serial Number 48E663FFFE30001F Last update Never

Dashboard

History

Downlinks

Configuration

Debug

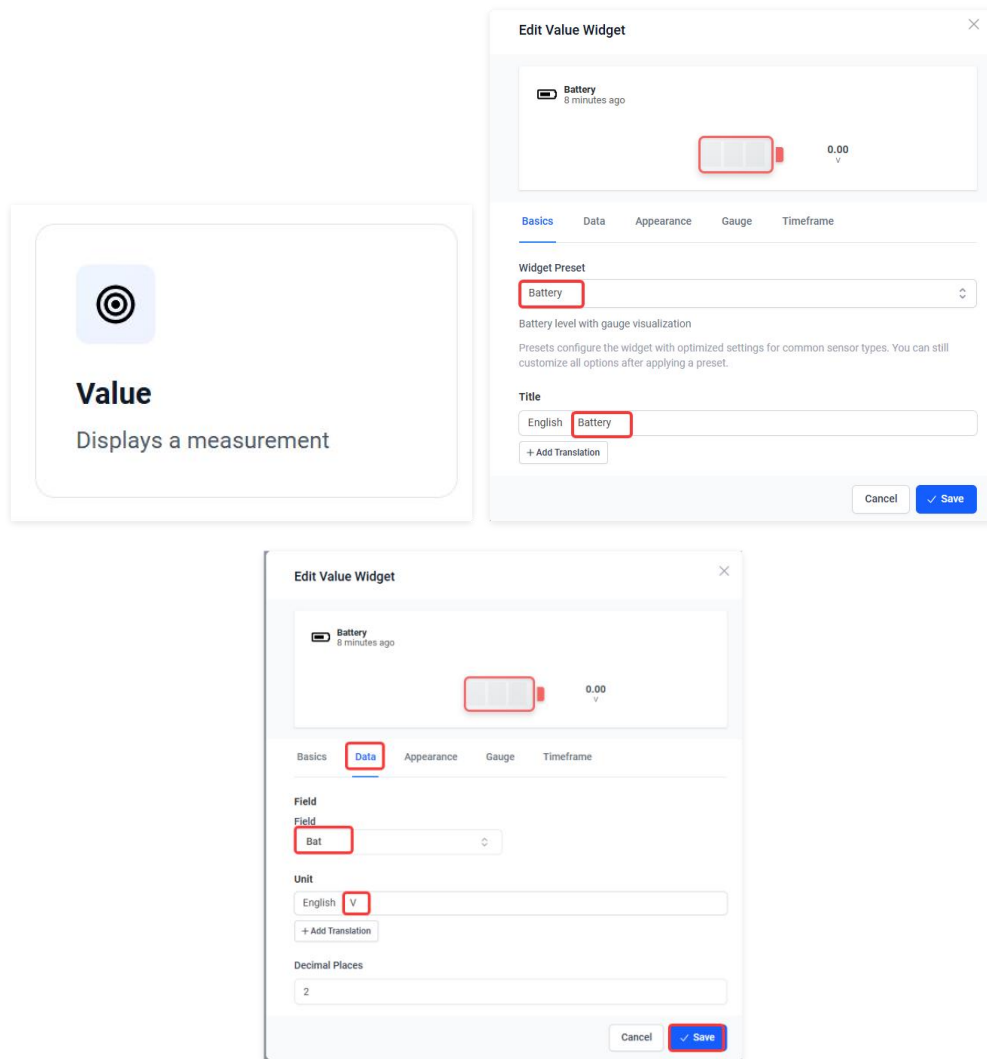
Rules

Permissions

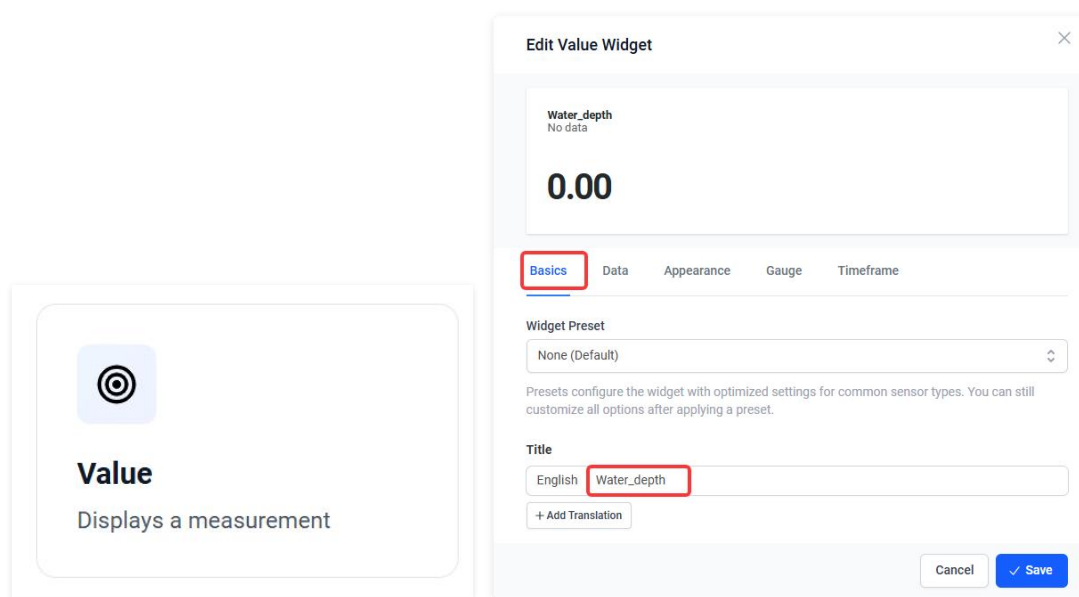
Public Link

+ Add Widget

13、 Add “Battery” field, Select “Value” and follow the steps below to complete the setup.



14、Select “Value” and set Title, Field and presentation form as well as the interval color.



Edit Value Widget

Water_depth
15 minutes ago

0.00
m

Basics **Data** Appearance Gauge Timeframe

Field
Field
Water Depth

Unit
English m

Decimal Places
2

Cancel Save

Basics Data Appearance **Gauge** Timeframe

Gauge Type
Gradient Ramp

Name	Values	Color
Range name	0	#ccc
Range name	1	#ed64a6
Range name	2	#ecc94b
Range name	3	#38b2ac
Range name	4	#667eea
Range name	5	#4299e1

Direction
☒ Left to Right ☐ Right to Left

Mode
☒ Ascending ☐ Descending

Cancel Save

15、Select “Value” and set Title, Field and presentation form as well as the interval color.

Value
Displays a measurement

Edit Value Widget

Water pressure
21 hours ago

0.00
Pa

Basics Data Appearance Gauge Timeframe

Widget Preset
None (Default)

Title
English Water pressure

Cancel Save

Edit Value Widget

Water pressure
21 hours ago

0.00
Pa

Basics **Data** Appearance Gauge Timeframe

Field
Field
Water Pressure

Unit
English Pa

Decimal Places
2

Cancel Save

Edit Value Widget

Water pressure
21 hours ago

0.00
Pa

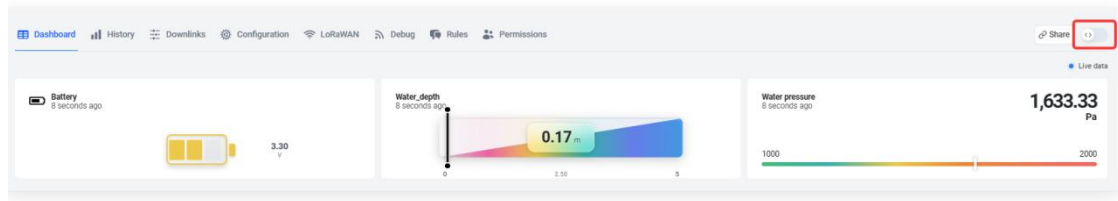
Basics Data Appearance **Gauge** Timeframe

Gauge Type
Linear

Name	Values	Color
Range name	1000	#48bb78
Range name	1200	#38b2ac
Range name	1400	#ecc94b
Range name	1600	#ed64a6
Range name	1800	#d89336
Range name	2000	#f56565

Cancel Save

16、Click the switch to save, and you can see the data visually.



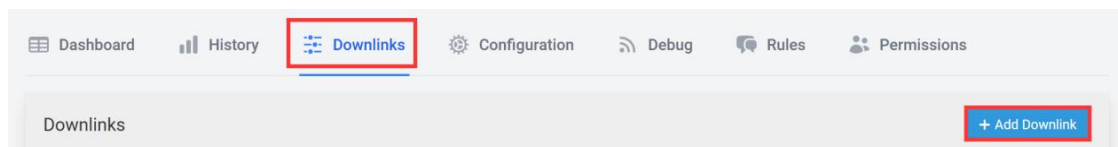
3.2.1 Downlink

- Modification time interval (Fport1)

Modify the time interval for uploading data, the default is one hour.

1、If you need to change time Interval (Default 60 minutes), you can click “Configuration-->Fields-->+Add Field”

2、Click “Downlink-->Add Downlink”.



Enter name、description、fields used and payload encoder respectively.

Name: Set User-Defined Sending Time Interval

Description: Set the user-defined report transmission interval and store it in the configuration variable.(5Min-1440Min)

Payload Encoder: copy in [Github](#).

Configure Downlink

Name
Set User-Defined Sending Time Interval

Description
Set the user-defined report transmission interval and store it in the configuration variable.(5Min-1440Min)

Fields used
If your encoder function takes input from the device's fields, you can specify them here. They will be used to create the form for the downlink generator.
SENDING_TIME_INTERVAL

☐ Trigger on measurements

If activated, each time the device records a measurement in one of the fields used, the downlink will be sent automatically.

Port
1

Payload Encoder

```

1 function Encoder(measurements, port) {
2   var interval = measurements["SENDING_TIME_INTERVAL"].value * 60;
3   if (interval < 300) {
4     interval = 300;
5     console.log("Interval < 300 Seconds / 5 Minutes not allowed!");
6   }
7   // Convert to hexadecimal only from interval
8   return interval.toString(16).padStart(4, '0').match(/.{2}/g).map(function(f) { return parseInt(f, 16);
9 });
10
11 // String.prototype.padStart() polyfill
12 // https://github.com/uzitten/polyfill/blob/master/string.polyfill.js
13 // https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/padStart
14 //
15 if (!String.prototype.padStart) {
16   String.prototype.padStart = function padStart(targetLength,padString) {
17     targetLength = targetLength>0; //truncate if number or convert non-number to 0;
18     padString = String(typeof padString !== 'undefined' ? padString : ' ');
19     if (this.length > targetLength) {
20       return String(this);
21     }
22     else {
23       targetLength = targetLength-this.length;
24       if (targetLength > padString.length) {
25         padString = padString.repeat(targetLength/padString.length); //append to original to en
26       }
27     }
28   }
29 }

```

3、Click “Dashboard-->switch-->+ Add Widget”.

Select “Downlink” and setting as follow image.

Edit Downlink Widget

User-Defined Time Interval(5Min-1440Min)

Basics | Data | Appearance

Title
English: User-Defined Time Interval(5Min-1440Min)
German: [dropdown]
+ Add Translation

Cancel | Save

Edit Downlink Widget

User-Defined Time Interval(5Min-1440Min)

Basics | **Data** | Appearance

Downlink
Set User-Defined Sending Time Interval

Additional Downlinks
+ Add

Cancel | Save

4、Click the switch to save, and you can click to change your time Interval.

User-Defined Time Interval(5Min-1440Min)

Sending Time Interval

1

Cancel | Save measurements and send downlink